



RELAZIONE DI PROGETTO

Patavium Open

Corso di Tecnologie Web

Anno Accademico 2025/2026

Componenti del Gruppo

Nominativo	Email Studentesca	Matricola
Marini Anna	anna.marini.7@studenti.unipd.it	2110986
Bellet Eleonora	eleonora.bellet@studenti.unipd.it	2111002
Belluz Diego	diego.belluz@studenti.unipd.it	2043361
Milanese Lorenzo	lorenzo.milanese.1@studenti.unipd.it	2117110

Responsabile di progetto: Marini Anna

Indirizzo Web del Progetto:

`http://` (*URL pubblico da inserire in seguito*)

Credenziali di Accesso

Amministratore	Username: <code>admin</code>	Password: <code>admin</code>
Utente	Username: <code>user</code>	Password: <code>user</code>

Indice

1	Origine del progetto	1
1.1	Patavium Open: un torneo fittizio con uno scopo reale	1
1.2	Utenti target	1
2	Architettura del Sito e Scelte Implementative	2
2.1	Struttura Logica e Flusso di Navigazione	2
2.1.1	Area Riservata e Transazionale	3
2.1.2	Area Amministrativa	3
2.2	Pattern Architetturale a Componenti (Template Engine)	3
2.3	Schema di navigazione dell'utente nel sito (metafora della pesca)	3
2.4	Pagine di Errore personalizzate	4
3	Accessibilità	5
3.1	Colori	5
3.2	Distinzione dei link visitati	7
3.3	Palette della Dark Mode	7
3.4	Validazione	7
4	Progettazione del Database	7
4.1	Schema Relazionale	9
4.2	Gestione degli Account	9
4.3	Sistema di Vendita: Programma, Ordini e Biglietti	9
4.4	Informazione e Supporto: News, FAQ e Domande	10
4.5	Corrispondenza tra immagini e testi alternativi	10
5	Dettagli implementativi	10
5.1	Markup HTML5 e Inclusività (Conformità WCAG 2.1 AA)	10
5.2	Logica di Backend e Gestione dei Dati (PHP)	11
5.2.1	Il Livello dei Controller (<code>php-pages</code>)	11
5.2.2	Il Livello dei Manager (<code>php-manager</code>)	12
5.2.3	Sicurezza Attiva e Validazione (La classe <code>Tool</code>)	12
5.2.4	Accessibilità generata Server-Side e Gestione Errori	12
5.3	Comportamento Client-Side e Interazioni Asincrone (JavaScript)	13
5.4	Sviluppo del Livello di Presentazione (CSS3 e Design System)	13
5.4.1	Design System e CSS Custom Properties	14
5.4.2	Dark Mode Architetturale	14
5.4.3	Accessibilità Visiva e Media Queries Avanzate	14
5.4.4	Approccio DRY e Raggruppamento dei Componenti	14
5.4.5	Ottimizzazione per la Stampa	14
6	Installazione e Configurazione del Sistema	15
6.1	Ambiente di sviluppo utilizzato	15
6.2	Procedura di installazione	15
7	Metodologia di Lavoro e Suddivisione dei Compiti	16
7.1	Pianificazione e Organizzazione	16
7.2	Suddivisione dello sviluppo	16
7.2.1	Fase 1: Struttura HTML e Database	16
7.2.2	Fase 2: Logica PHP e JavaScript	16
7.3	Strumenti di comunicazione	17

7.4	Monitoraggio dei Task: La “Task Board” su Google Doc	18
-----	--	----

1 Origine del progetto

Abbiamo ideato il progetto **Patavium Open** unendo le competenze apprese durante il corso di Tecnologie Web al nostro comune interesse per il tennis. Nel corso degli ultimi anni, durante la frequentazione di tornei (sia dal vivo che in preparazione di eventuali acquisti di biglietti), ci siamo spesso imbattuti in siti web ufficiali di eventi tennistici che presentavano criticità non trascurabili dal punto di vista dell'usabilità e dell'accessibilità.

In particolare, abbiamo osservato ripetutamente:

- difficoltà nel reperire informazioni chiare su come e dove acquistare i biglietti;
- strutture dei menù poco intuitive, che nascondevano i prezzi, i tipi di abbonamento e i servizi inclusi;
- assenza di un percorso logico che guidasse l'utente, soprattutto il meno esperto, verso la scelta del biglietto o dell'abbonamento più adatto;
- pagine sovraccariche di elementi grafici e testi promozionali, ma povere di dati strutturati (date, campi di gioco, modalità di accesso).

Questi ostacoli, sebbene apparentemente minori, si traducono in una perdita di tempo reale per l'utente finale: ore spese a cercare conferme, a interpretare menù ambigui o a chiedere supporto su forum esterni. Di conseguenza, abbiamo ritenuto utile mettere le nostre competenze al servizio di un progetto dimostrativo che rispondesse a tali lacune.

1.1 Patavium Open: un torneo fittizio con uno scopo reale

Abbiamo quindi immaginato un torneo di tennis internazionale che si svolgesse idealmente a Padova, battezzandolo **Patavium Open** (dal nome latino della città, *Patavium*). L'obiettivo non era realizzare il sito di un evento esistente, ma **progettare e sviluppare un prototipo web che risolvesse in modo concreto i problemi di chiarezza, accessibilità e reperibilità dei contenuti** riscontrati nei siti reali.

Il sito del Patavium Open si propone quindi di:

- esporre in modo esplicito e gerarchico tutte le informazioni sui biglietti e sugli abbonamenti;
- guidare l'utente nell'acquisto con pochi passaggi ben identificabili;
- garantire l'accesso ai contenuti anche a persone con disabilità o con dispositivi e connessioni limitate;
- separare nettamente contenuto, presentazione e comportamento (HTML, CSS, PHP/JS) come richiesto dalle specifiche del corso.

1.2 Utenti target

L'utenza primaria è costituita da **appassionati di tennis** di qualsiasi età e competenza tecnologica: dai giovani che seguono il circuito ATP/WTB, ai genitori che accompagnano ragazzi ai tornei, fino agli spettatori occasionali. Tutti questi utenti condividono l'esigenza di:

- trovare rapidamente le date e il luogo del torneo;
- confrontare le opzioni di biglietteria (giornaliero, settimanale, tribuna centrale, campo secondario, eventuali pacchetti famiglia);
- completare l'acquisto con un flusso controllato (convalida lato client e lato server, salvataggio in database).

Abbiamo scelto di non includere funzionalità “di facciata” (like, commenti, social network) se non strettamente necessarie, per concentrarci invece sugli aspetti che generano maggiore frustrazione nei siti esistenti: **chiarezza e controllabilità dell’interazione**.

Nella progettazione abbiamo tenuto conto anche di possibili motori di ricerca (SEO): il sito deve rispondere a query tipiche come “*Patavium Open biglietti*”, “*quando si gioca il torneo di Padova*”, “*prezzi abbonamenti tennis Padova*”. Le azioni intraprese per migliorare il ranking vengono discusse nella sezione dedicata dell’analisi.

2 Architettura del Sito e Scelte Implementative

L’architettura del portale *Patavium Open* è stata concepita per guidare l’utente attraverso un flusso di navigazione intuitivo, adottando un approccio modulare. Il sistema separa in modo netto la logica di business dall’interfaccia utente, organizzando le risorse in directory specializzate e standardizzando i nomi dei file secondo la convenzione `snake_case` per garantire pulizia e compatibilità.

2.1 Struttura Logica e Flusso di Navigazione

Area Pubblica e Informativa

Accessibile a tutti, è la vetrina principale del portale e immerge subito l’utente nel contesto del tennis professionistico:

- **Home Page (`pages/index.html`):** Il fulcro del sito. Offre una panoramica immediata sull’evento, le notizie in evidenza e un accesso rapido alle azioni principali (come l’acquisto dei biglietti).
- **Sezione Biglietteria (`php-pages/biglietti.php`):** Presentata tramite card chiare e descrittive, divide l’offerta in tre opzioni pensate per le reali esigenze del pubblico:
 - *Abbonamento (`php-pages/abbonamento.php`):* Accesso completo per l’intera durata del torneo.
 - *Ground Pass (`php-pages/ground_passes.php`):* Titolo d’ingresso per vivere l’atmosfera del parco sportivo, accedere alle aree ristoro e seguire i match dai maxischermi, senza un posto numerato negli stadi principali.
 - *Singola Sessione (`php-pages/single_session.php`):* Acquisto flessibile e mirato per la sessione diurna o serale di una specifica giornata.
- **News (`php-pages/news.php`):** Modulo dinamico per gli aggiornamenti, i risultati in tempo reale e gli approfondimenti. Mantiene vivo l’interesse del pubblico e favorisce un’ottima indicizzazione sui motori di ricerca.
- **Supporto Interattivo, le FAQ (`php-pages/faq.php`):** Molto più di un semplice elenco statico. Se un utente non trova la risposta che cerca tra le domande frequenti, può inviare un quesito personalizzato. Questo crea un vero sistema di ticketing: la domanda arriva alla dashboard dell’amministratore, che può rispondere facendo apparire la notifica direttamente nell’area personale dell’utente.
- **Informativa Legale (`php-pages/termini.php`, `php-pages/privacy.php`):** Pagine dedicate alla trasparenza, logicamente separate dal supporto, dove consultare i termini di servizio e le policy sulla privacy.

2.1.1 Area Riservata e Transazionale

Le funzionalità interattive e legate agli acquisti sono protette da autenticazione:

- **Autenticazione (`php-pages/login.php`, `php-pages/registrazione.php`):** Pagine che gestiscono l'accesso e la creazione sicura degli account.
- **Carrello (`php-pages/carrello.php`):** Il cuore delle transazioni. L'utente gestisce i biglietti scelti, visualizza il riepilogo dinamico dei costi e conclude l'acquisto.
- **Area Utente (`php-pages/area_utente.php`):** La dashboard personale. Qui l'utente ritrova lo storico dei suoi ordini, i ticket generati e un centro notifiche dove arrivano le risposte dell'amministrazione ai quesiti posti nella sezione FAQ.

2.1.2 Area Amministrativa

- **Gestione Torneo (`php-pages/area_admin.php`):** Pannello di controllo riservato allo staff, da cui l'amministratore gestisce in modo completo (inserimento, modifica ed eliminazione) tutti i contenuti dinamici del portale. In particolare può:
 - pubblicare, aggiornare ed eliminare le **notizie** (*News*), scegliendo anche quali mettere in evidenza in Home Page;
 - gestire l'**Albo dei Campioni**, ossia i ritratti dei vincitori delle passate edizioni, con categoria, anno e ordine di visualizzazione;
 - creare e modificare le **FAQ** ufficiali, suddivise per categoria;
 - **rispondere ai quesiti** inviati dagli utenti tramite il sistema di ticketing, facendo comparire la notifica direttamente nell'area personale dell'utente.

L'intero pannello garantisce così che il sito sia sempre aggiornato e interattivo.

2.2 Pattern Architetturale a Componenti (Template Engine)

Per rispettare il principio DRY (Don't Repeat Yourself) ed evitare codice ridondante, l'interfaccia non è stata costruita con file monolitici, ma tramite un motore di template custom lato server gestito dal PHP. Il sistema sfrutta due cartelle principali:

- **La directory `pages/`:** Contiene i layout macroscopici. Questi file HTML definiscono la struttura di base e posizionano dei segnaposto (come `[Header]` o `[lista_ordini]`) in attesa dei dati.
- **La directory `item/`:** Contiene i *Template Fragments*. Elementi grafici che si ripetono (come la card di un biglietto, la riga di una notifica o il blocco di una notizia) sono stati isolati qui in singoli file HTML.

Grazie a questa architettura, quando bisogna mostrare una lista di notizie, il PHP recupera i dati dal database, riempie il frammento HTML prelevato da `item/` per ogni singola news, e inietta il risultato finale nella pagina. Se in futuro vorremo modificare il design di una card, basterà aggiornare un solo file per vederlo cambiato in tutto il sito.

2.3 Schema di navigazione dell'utente nel sito (metafora della pesca)

Il portale web del Patavium Open presenta un'architettura dell'informazione solida e strutturata, concepita per assecondare i molteplici comportamenti di ricerca tipici degli utenti. Possiamo analizzare l'attuale configurazione del sito attraverso la "metafora della pesca":

- **Tiro perfetto:** Per ottimizzare l'esperienza degli utenti *goal-oriented* (ovvero coloro che accedono al portale con un intento ben preciso e sanno esattamente cosa cercare) sono state implementate specifiche soluzioni per velocizzare la navigazione. Da un lato, per intercettare l'utente pronto all'acquisto dei biglietti, è stata inserita una *Call to Action* primaria e ben visibile (il pulsante **“Acquista i tuoi biglietti”**) direttamente nella sezione *hero* della *Home Page*, riducendo al minimo i clic necessari per accedere all'offerta. Dall'altro lato, per supportare la ricerca di informazioni operative e logistiche (quali dettagli su parcheggi, varchi di ingresso, orari dei match o regolamenti), la pagina dedicata alle **FAQ** è stata potenziata con una barra di ricerca reattiva. Questa funzionalità permette di filtrare dinamicamente le risposte a schermo in base alle parole chiave digitate, garantendo un recupero immediato e mirato delle informazioni senza costringere l'utente a scorrere estesi elenchi testuali.
- **Trappola per aragoste:** La struttura si adatta perfettamente anche a chi naviga senza un'idea predefinita, attirandolo e guidandolo progressivamente. Il flusso di acquisto nella sezione Biglietti ne è la dimostrazione pratica: l'utente entra con un'intenzione generica e viene incanalato, come in una trappola a cilindri concentrici, verso scelte sempre più specifiche (seleziona prima la macro-tipologia tra Abbonamento, Ground Passes o Singola Sessione, per poi scendere nel dettaglio del posto e della giornata).
- **Pesca con la rete:** L'esigenza di esplorare un argomento in tutta la sua totalità è soddisfatta dalla sezione News. Questo ambiente è progettato per gli utenti che non vogliono perdersi nulla, offrendo una panoramica aggregata di tutti gli aggiornamenti e i risultati, permettendo di "catturare" con un'unica mossa tutta l'attualità legata al torneo.
- **Boa di segnalazione:** Il sistema di carrello è stato progettato per garantire la massima persistenza e continuità durante l'esperienza di navigazione. Abbandonando un approccio puramente client-side, la gestione dello stato è stata centralizzata lato server mediante l'utilizzo dell'array `$_SESSION`. Questa scelta architetturale assicura che gli articoli selezionati non vadano persi durante il cambio di pagina o i ricaricamenti. Dal punto di vista dell'interfaccia, l'icona del carrello (costantemente visibile nell'header globale) funge da punto di ancoraggio per l'utente: ciò gli consente di interrompere temporaneamente il flusso di acquisto per esplorare altre sezioni del sito o consultare materiali di supporto, con la totale sicurezza di poter riprendere e finalizzare la transazione in qualsiasi momento, senza alcuna perdita del contesto operativo.

2.4 Pagine di Errore personalizzate

Per garantire un'esperienza utente fluida ed evitare che i visitatori si sentano disorientati in caso di imprevisti, abbiamo progettato delle pagine di errore personalizzate per i codici HTTP più comuni (403, 404 e 500).

Dal punto di vista dell'infrastruttura di backend, il reindirizzamento automatico a queste schermate è gestito a livello di server tramite il file di configurazione `.htaccess`. Sfruttando la direttiva `ErrorDocument` di Apache, il sistema intercetta nativamente gli stati di errore HTTP e instrada l'utente verso i rispettivi file HTML personalizzati, impedendo la visualizzazione dei messaggi standard del server che risulterebbero estranei al contesto dell'applicazione.

In questo modo, anche in situazioni di errore le pagine mantengono intatto il *Design System* del sito (inclusi *header*, *footer* e il percorso *breadcrumb*) e utilizzano un tono di voce ironico e coerente con il tema del torneo. Ogni errore è infatti associato a una metafora tennistica e accompagnato da un'immagine dedicata (una pallina da tennis "triste"), offrendo sempre una chiara *Call to Action* per tornare rapidamente alla *Home Page*:

- **Errore 403 (Accesso Negato):** Accompagnato dal messaggio *"Area riservata ai giocatori!"*, spiega all'utente che sta tentando di accedere a una risorsa per cui non ha i permessi

necessari (come il pannello di amministrazione senza un login valido).

- **Errore 404 (Pagina non trovata):** Introdotto dalla frase *"Palla fuori campo!"*, rassicura l'utente nel caso in cui abbia digitato un indirizzo errato o cercato una risorsa non più disponibile nel server.
- **Errore 500 (Errore interno del server):** Segnalato come *"Interruzione per pioggia!"*, informa in modo controllato che si è verificato un problema tecnico lato applicazione e che lo staff è già al lavoro per ripristinare il corretto funzionamento del sistema.

Anche in queste pagine secondarie abbiamo mantenuto un forte rigore per l'accessibilità: gli elementi puramente visivi e decorativi (come i grandi numeri di errore sullo sfondo e l'immagine della pallina triste) sono stati silenziati per le tecnologie assistive tramite l'attributo `aria-hidden="true"`, mentre gli *skip-link* rimangono attivi per consentire di saltare i blocchi ripetitivi e raggiungere subito il testo informativo.

3 Accessibilità

3.1 Colori

La seguente sezione illustra la palette cromatica adottata per l'interfaccia utente del progetto. I colori sono stati rigorosamente selezionati per bilanciare l'identità visiva dello scopo del sito (scegliendo colori che ricordassero la terra rossa e il suolo blu dei campi da tennis) con i fondamentali requisiti di accessibilità.

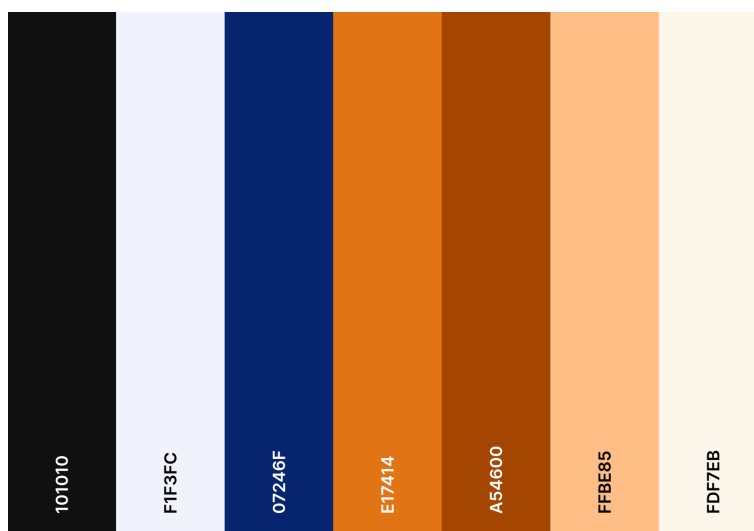


Figure 1: La palette di colori del sito, con i relativi codici esadecimali.

La seguente tabella riporta l'analisi dei contrasti cromatici per le combinazioni di colore utilizzate nel foglio di stile (CSS), indicando la conformità ai criteri di accessibilità WCAG (Web Content Accessibility Guidelines) e i relativi contesti di applicazione nel layout.

Per testare i contrasti tra i colori abbiamo utilizzato il sito: **Tanaguru Contrast-Finder** (<https://contrast-finder.tanaguru.com/>), che fornisce un'analisi dettagliata dei rapporti di contrasto e verifica la conformità ai livelli WCAG 2.1 (AA e AAA) per i testi normali e grandi.

Colore 1	Colore 2	Contrasto	Livello WCAG	Dove viene usato nel CSS
#F1F3FC	#101010	17.18	AAA	È il contrasto principale del sito. Usato per il testo base (body), le briciole di pane (#breadcrumb), e i testi interni alle card e ai form.
#07246F	#F1F3FC	12.72	AAA	Usato per i titoli (H1, H2) sul colore di sfondo principale. Ad esempio nella .sezione-benvenuto, nei .titoli-notizie e nei form.
#07246F	#FDF7EB	13.20	AAA	Usato per i testi secondari sopra gli sfondi blu scuro del brand. Si trova nel footer (.footer-tagline, contatti) e nel testo delle card delle notizie.
#07246F	#FFBE85	8.68	AAA	Usato per gli stati attivi (hover, focus, link correnti) sopra gli sfondi brand. Esempi: .current-link nel menu, l'orario delle sessioni (.session-time).
#A54600	#FDF7EB	5.66	AA (AAA per testi grandi)	Usato per i link o le azioni posizionati sulla "superficie alternativa" (es. testi nella .register-section o messaggi di attenzione).
#A54600	#F1F3FC	5.45	AA (AAA per testi grandi)	Usato per i testi link e le call to action sul colore di sfondo principale. Esempio: i testi in .contatti-sezione o i link dei breadcrumb.
#07246F	#E17414	4.50	AA	Usato per i bordi decorativi e i separatori sulle sezioni brand. Esempio: il bordo superiore del footer (.footer-sito) e l'icona delle sessioni.
#FDF7EB (Link non visitato)	#FFBE85 (Link visitato)	1.51	Non superato*	Si verifica nel footer (.footer-col ul a vs a:visited) e nell'header. Differenzia gli stati del link tra di loro sul colore di sfondo brand.

Table 1: Analisi dei rapporti di contrasto e mappatura delle classi CSS

3.2 Distinzione dei link visitati

Per i link già visitati abbiamo scelto un colore arancione chiaro (`#FFBE85`). Come si nota nell'ultima riga della Tabella 1, questo arancione ha però un contrasto molto basso (1.51) rispetto al bianco caldo usato per i link non ancora visitati. Distinguere i due stati basandosi solo sul colore avrebbe quindi violato il criterio WCAG 2.1 numero 1.4.1 (*Use of Color*), che impone di non affidare mai un'informazione al solo colore: una scelta che penalizzerebbe soprattutto gli utenti con difficoltà nella percezione cromatica.

Per superare questo limite, accanto a ogni link già visitato compare una piccola **icona a forma di pallina da tennis**. In questo modo l'utente si accorge di aver già aperto quella pagina anche senza distinguere bene il colore, perché entra in gioco un secondo indizio visivo, cioè la forma. L'icona, oltre a richiamare il tema del torneo, è immediata e facile da riconoscere.

* È proprio per questo motivo che, pur non raggiungendo la soglia di contrasto WCAG prevista per i testi normali (4.5:1), la combinazione di colori dei link resta accettabile: lo stato del link non viene comunicato dal solo colore. Va inoltre osservato che il colore del link visitato e quello del link non visitato, presi singolarmente rispetto allo sfondo, rispettano comunque il requisito AA con un contrasto superiore a 4.5:1.

Sul piano implementativo (file `visited-links.js`) la soluzione non sfrutta la pseudo-classe CSS `:visited`: per motivi di privacy i browser ne consentono soltanto il cambio di colore e non la rendono leggibile da JavaScript, quindi non permettono di aggiungere icone. Abbiamo perciò realizzato un tracciamento nostro, che salva in `localStorage` le pagine aperte sul sito e assegna la classe `.link-visitato` ai link di header, footer e contenuto che puntano a quelle pagine. È questa classe, e non `:visited`, a far comparire la pallina tramite CSS. La pallina è puramente decorativa (viene disegnata con uno pseudo-elemento `::after`) e quindi non viene letta dagli screen reader. Abbiamo volutamente evitato di aggiungere un testo nascosto per annunciare lo stato visitato: sarebbe stato ridondante, perché i browser comunicano già di loro agli screen reader, sulla base della cronologia, se un link è stato visitato, e un testo in più non farebbe che duplicare l'annuncio appesantendo la lettura assistita.

3.3 Palette della Dark Mode

Il portale offre anche una modalità scura (*Dark Mode*). La figura seguente ne riporta la palette: per assicurare un'esperienza coerente e accessibile anche al buio, abbiamo scelto tonalità che mantengono un contrasto adeguato e seguono le stesse linee guida WCAG 2.1, adattando i colori in modo da conservare comunque l'identità visiva del brand.

3.4 Validazione

Al fine di verificare la correttezza del codice e garantire un elevato livello di accessibilità, abbiamo sottoposto le pagine del sito a validazione. L'analisi è stata condotta utilizzando due differenti strumenti, ottenendo in entrambi i casi risultati positivi:

- **Lighthouse**: i test hanno confermato l'ottima qualità del codice, assegnando alle pagine un punteggio di accessibilità pari al 100%.
- **Total Validator**: le pagine hanno superato la validazione senza presentare alcun errore; lo strumento si è limitato a segnalare esclusivamente alcuni warning.

4 Progettazione del Database

Per la gestione dei dati del portale abbiamo optato per un database relazionale (MySQL/MariaDB), progettato partendo dalla definizione del modello Entità-Relazione (ER). L'obiettivo principale

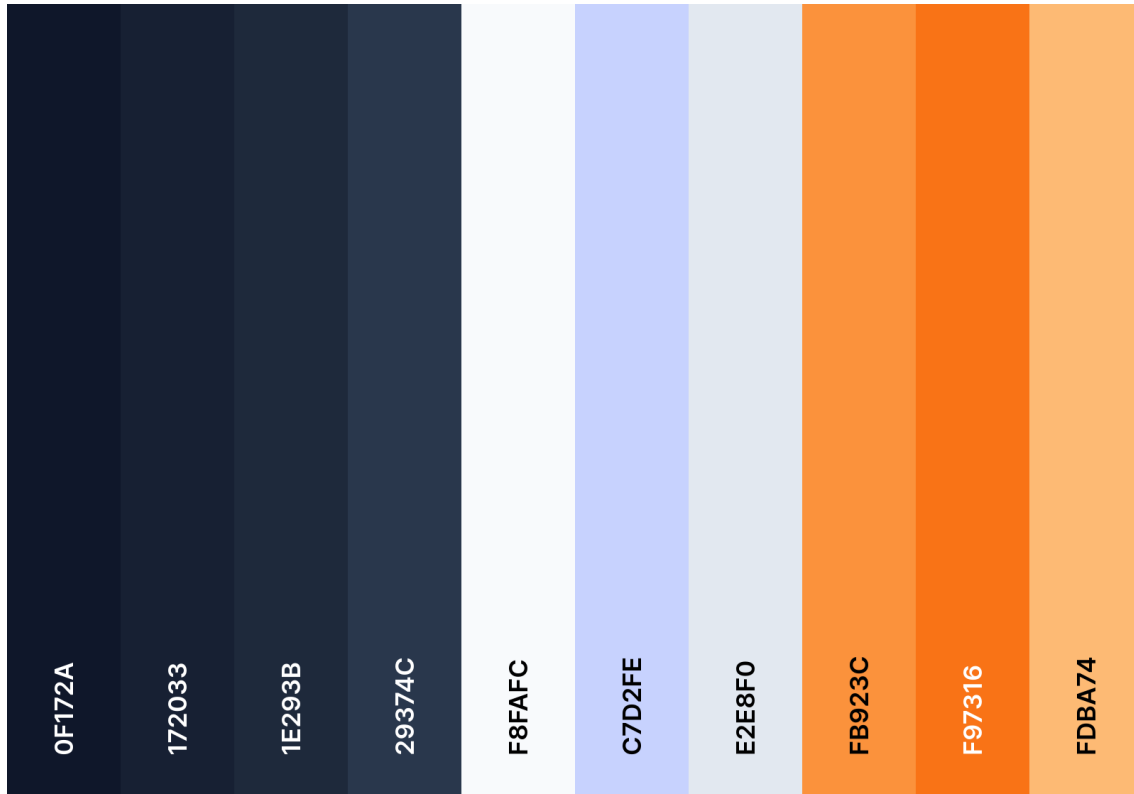


Figure 2: La palette di colori della dark mode, con i relativi codici esadecimali.

è stato mantenere il database normalizzato, evitando la ridondanza delle informazioni e garantendo l'integrità dei dati tramite l'uso rigoroso delle chiavi esterne.

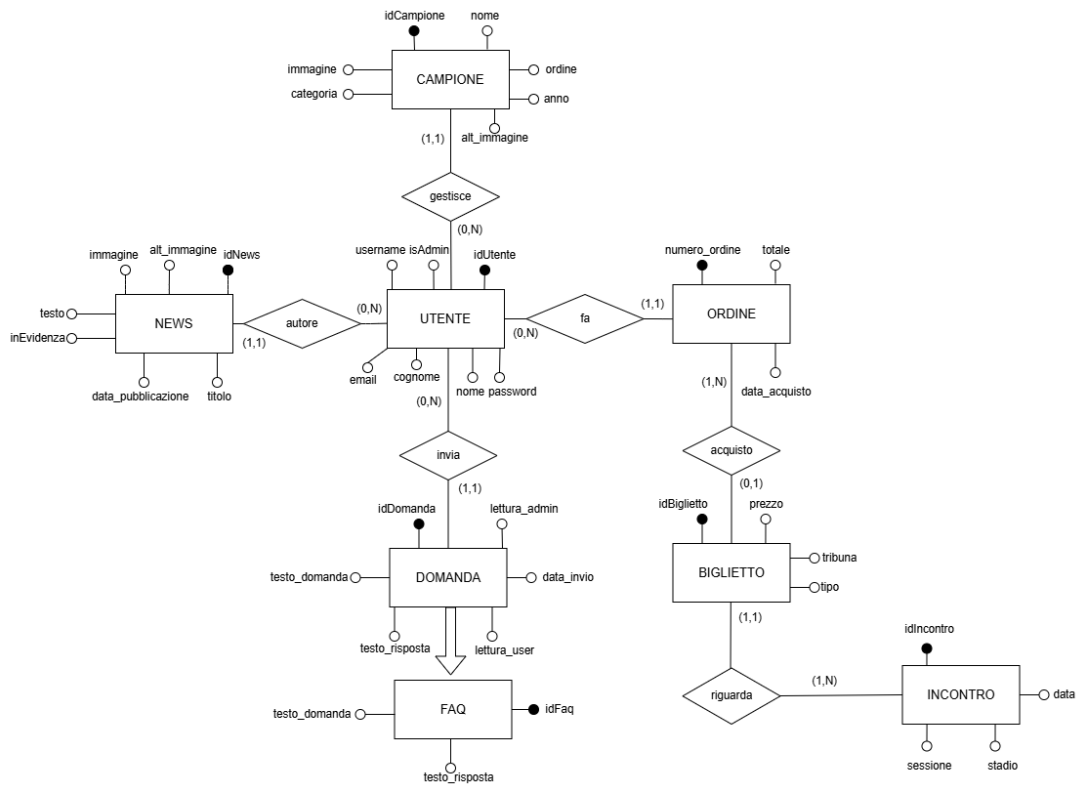


Figure 3: Schema E-R del database Patavium Open

4.1 Schema Relazionale

Di seguito è riportato lo schema logico relazionale derivato dal modello concettuale. Le relazioni presentano le chiavi primarie indicate con la sottolineatura, mentre gli attributi opzionali, che ammettono il valore NULL, sono contrassegnati da un asterisco (*).

- **UTENTE**(idUtente, username, email, password, nome, cognome, isAdmin)
- **INCONTRO**(idIncontro, data, sessione*, stadio)
- **NEWS**(idNews, titolo, testo, data_pubblicazione, immagine, alt_immagine, idAutore, inEvidenza)
FK: idAutore → **UTENTE**(idUtente)
- **DOMANDA**(idDomanda, testo_domanda, testo_risposta*, lettura_admin, lettura_user, data_invio, idUtente)
FK: idUtente → **UTENTE**(idUtente)
- **ORDINE**(numero_ordine, totale, data_acquisto, idUtente)
FK: idUtente → **UTENTE**(idUtente)
- **BIGLIETTO**(idBiglietto, prezzo, tribuna*, tipo*, numero_ordine*, idIncontro)
FK: numero_ordine → **ORDINE**(numero_ordine)
FK: idIncontro → **INCONTRO**(idIncontro)
- **FAQ**(idFaq, testo_domanda, testo_risposta, categoria)
- **CAMPIONE**(idCampione, nome, categoria, anno, immagine, alt_immagine, ordine)

Come si può notare dallo schema in Figura 3, il database è strutturato in tre macro-aree logiche: la gestione degli account, l'area di e-commerce (ordini e biglietti) e l'area di interazione e supporto (news e ticket).

4.2 Gestione degli Account

Il cuore del sistema è la tabella **UTENTE**. Al suo interno salviamo i dati anagrafici e le credenziali di accesso. Per questioni di sicurezza, le password non vengono mai salvate in chiaro nel database, ma sono cifrate utilizzando la funzione nativa di hashing **bcrypt** di PHP.

Invece di creare due tabelle separate per i clienti e per gli amministratori, abbiamo preferito semplificare la struttura inserendo un campo booleano chiamato **isAdmin**. Se questo flag è a 1, il sistema riconosce l'utente come membro dello staff e gli sblocca l'accesso al pannello di controllo (**area_admin.php**); altrimenti, l'utente naviga con i privilegi standard.

4.3 Sistema di Vendita: Programma, Ordini e Biglietti

La gestione della biglietteria è stata progettata per prevenire alla radice problemi di concorrenza o di *overbooking* (ad esempio, quando due utenti tentano di comprare lo stesso posto nello stesso istante). Per farlo, utilizziamo tre tabelle interconnesse:

- **INCONTRO**: Contiene il calendario del torneo. Ogni riga rappresenta un evento, definendone la data, lo stadio (es. Patavium Arena o Giotto Court) e il tipo di sessione (diurna, serale o ground).
- **BIGLIETTO**: La nostra scelta implementativa è stata quella di "pre-generare" fisicamente nel database tutti i biglietti disponibili per ogni partita, ancor prima che vengano venduti. Ogni biglietto nasce con un prezzo, una tribuna e il collegamento all'incontro, ma con il campo **numero_ordine** impostato a NULL.

- **ORDINE:** Quando un utente completa un acquisto dal carrello, il sistema crea un record in questa tabella (registrando il totale pagato e la data) e va semplicemente a inserire l'ID di questo nuovo ordine dentro i biglietti scelti. Se un biglietto ha il `numero_ordine` valorizzato, il sistema sa che è già stato venduto e non lo mostra più tra quelli disponibili.

4.4 Informazione e Supporto: News, FAQ e Domande

L'interazione con gli utenti e l'aggiornamento del portale si basano su tre tabelle specifiche:

- **NEWS:** Tabella dedicata agli articoli e agli aggiornamenti. È collegata all'utente amministratore che l'ha scritta (tramite `idAutore`) e include i campi per il percorso dell'immagine e per il suo testo alternativo (`alt_image`), fondamentale per l'accessibilità. Il flag `inEvidenza` è molto utile per permettere all'admin di scegliere quali notizie far comparire in primo piano nella Home Page.
- **FAQ:** Una tabella indipendente e di consultazione pubblica, che contiene semplicemente le domande frequenti e le relative risposte.
- **DOMANDA:** Rappresenta il vero e proprio sistema di supporto clienti (Ticketing). Qualsiasi utente registrato può inviare un quesito. Al momento dell'invio, il sistema compila il `testo_domanda` e lascia temporaneamente vuoto il `testo_risposta`. Per gestire le notifiche in modo efficiente, abbiamo inserito due flag: `lettura_admin` e `lettura_user`. Quando l'utente scrive, il sistema avvisa l'amministratore; non appena l'admin risponde, il flag dell'utente scatta, facendogli comparire la notifica visiva direttamente all'interno della sua Area Personale, in modo che sappia di aver ricevuto assistenza.
- **CAMPIONE:** Tabella dedicata all'Albo dei Campioni delle passate edizioni del torneo, gestibile dall'amministratore. Oltre ai dati anagrafici del vincitore (`nome`, `categoria`, `anno`) e al percorso dell'immagine, prevede il campo `ordine` per controllare la sequenza di visualizzazione e, come per le news, il campo `alt_image` per la descrizione testuale del ritratto.

4.5 Corrispondenza tra immagini e testi alternativi

Una scelta progettuale trasversale, motivata da ragioni di accessibilità, è stata quella di riservare **a ogni immagine caricata dinamicamente un proprio campo dedicato per il testo alternativo** (`alt_image`) direttamente nel database: è presente sia nella tabella **NEWS** sia nella tabella **CAMPIONE**. In questo modo la descrizione testuale viaggia sempre insieme alla relativa immagine ed è gestita dall'amministratore nello stesso momento in cui carica il file, garantendo una corrispondenza permanente e non casuale tra contenuto visivo e contenuto testuale. Per rafforzare questa garanzia, le colonne `alt_image` sono definite direttamente nello schema (`patavium_open.sql`) come NOT NULL con un valore di *default* sensato, così da impedire a livello di schema l'esistenza di un'immagine priva di alternativa testuale.

5 Dettagli implementativi

5.1 Markup HTML5 e Inclusività (Conformità WCAG 2.1 AA)

Particolare attenzione è stata posta alla stesura del codice HTML, considerato non come semplice strumento di impaginazione, ma come livello semantico fondamentale per l'accessibilità e l'indicizzazione. Le interfacce sono state sviluppate rispettando rigorosamente le linee guida WCAG 2.1 livello AA, attraverso l'adozione dei seguenti pattern:

- **HTML5 Strict e Graceful Degradation:** Il markup è stato scritto in rigoroso HTML5, prestando attenzione a mantenere una sintassi di tipo XML (chiusura esplicita di tutti i tag e attributi correttamente quotati). Il sito è stato progettato per "degradare con grazia": anche in assenza di fogli di stile o con JavaScript disabilitato, il contenuto rimane perfettamente leggibile, gerarchico e navigabile grazie al solo HTML puro.
- **Landmark Semantici e Segmentazione:** La struttura di ogni pagina evita l'abuso di contenitori generici (`<div>`), affidandosi a tag semantici nativi (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`). Questo partizionamento logico permette agli screen reader (come NVDA o JAWS) di generare un albero di navigazione corretto, consentendo agli utenti con disabilità visive di orientarsi immediatamente all'interno dei contenuti.
- **Liste di Definizione Semantiche:** Per la presentazione di dati strutturati (come i dettagli dei match o le specifiche dei biglietti), si è optato per l'utilizzo delle *Definition Lists* (`<dl>`, `<dt>`, `<dd>`). Questa scelta architetturale crea un legame logico indissolubile tra l'etichetta (es. "Data:") e il suo valore, garantendo una gerarchia informativa impeccabile e semanticamente superiore rispetto a semplici paragrafi affiancati.
- **Feedback Asincroni e Live Regions:** Per garantire un'esperienza utente reattiva e accessibile, la validazione dei form (es. Login e Registrazione) fa uso delle *ARIA Live Regions*. Implementando l'attributo `aria-live="polite"`, il sistema annuncia in tempo reale i messaggi di errore o di successo, assicurando che l'utente venga notificato in modo vocale senza subire disorientanti ricaricamenti di pagina o perdite di focus.
- **Skip-Link e Gestione del Focus:** Per favorire la navigazione tramite tastiera, è stato integrato uno *Skip Link* ("Vai al contenuto") visibile solo al momento della ricezione del focus. Inoltre, tutti gli elementi interattivi presentano indicatori visivi di focus (*outline*) studiati per mantenere un elevato rapporto di contrasto sia nel tema chiaro che nella Dark Mode.
- **Contenuti Multimediali e Decorativi:** Le immagini informative sono provviste di attributi `alt` concisi. Al contrario, gli elementi puramente estetici (come file vettoriali SVG o icone di accompagnamento) sono stati silenziati tramite gli attributi `alt=""` e `aria-hidden="true"`, al fine di non generare "rumore cognitivo" durante la lettura assistita.
- **Utilità per Screen Reader:** Per menu a tendina o pannelli espandibili sono stati usati i tag nativi `<details>` e `<summary>`, che garantiscono nativamente l'accessibilità da tastiera. Infine, informazioni visivamente superflue ma necessarie al contesto sono state preservate per gli screen reader utilizzando la classe di utilità CSS `.sr-only` (che nasconde l'elemento a schermo senza rimuoverlo dal DOM) e l'attributo `lang="en"` per evitare errori di pronuncia sui termini inglesi.

5.2 Logica di Backend e Gestione dei Dati (PHP)

Il cuore del sistema è stato sviluppato in PHP puro, adottando un pattern architetturale molto vicino al modello MVC (Model-View-Controller). L'obiettivo primario è stato separare la gestione delle richieste HTTP dalla logica di interazione con il database. Per farlo, abbiamo diviso gli script in due macro-directory: `php-pages` e `php-manager`.

5.2.1 Il Livello dei Controller (`php-pages`)

I file contenuti in questa cartella (come `abbonamento.php` o `area_admin.php`) fungono da orchestratori. Il loro compito è intercettare le richieste dell'utente (tramite GET o POST), verificare

in modo stringente i permessi di sessione e dialogare con i Manager per ottenere i dati necessari. Una volta recepite le informazioni, il controller sfrutta il nostro motore di template custom per iniettarle all'interno dei file HTML, per poi restituire la pagina finita al browser. Abbiamo inoltre gestito le chiamate asincrone (AJAX), permettendo ai controller di interrompere l'esecuzione e restituire risposte pulite in formato JSON per le interazioni dinamiche dell'interfaccia.

5.2.2 Il Livello dei Manager (php-manager)

Tutta la logica di business e l'interazione con il database relazionale sono state isolate nei file Manager. Classi come `TicketManager` e `NewsManager` incapsulano interamente le query SQL. Abbiamo dedicato molta cura alla stesura di query complesse e ottimizzate: ad esempio, il calcolo della disponibilità reale degli abbonamenti non è un dato statico, ma viene elaborato al volo incrociando i biglietti invenduti con i raggruppamenti delle 14 sessioni del torneo. Questo isolamento garantisce che un'eventuale futura modifica alla struttura del database richiederà aggiornamenti solo all'interno dei Manager, lasciando intatto il resto del sito.

5.2.3 Sicurezza Attiva e Validazione (La classe Tool)

La sicurezza è stata trattata come requisito fondamentale e non come aggiunta a posteriori. Abbiamo centralizzato tutte le funzioni di utilità e validazione nella classe statica `Tool`:

- **Prevenzione XSS (Cross-Site Scripting):** Qualsiasi input inserito dall'utente viene ripulito tramite il metodo `Tool::pulisciInput()`. Questa funzione combina `strip_tags` e `htmlspecialchars` con i flag più restrittivi (`ENT_QUOTES | ENT_HTML5`), garantendo che codice malevolo o script iniettati nei form vengano neutralizzati prima di arrivare al database.
- **Prevenzione SQL Injection:** Utilizziamo ovunque i **Prepared Statements** (`bind_param`), delegando al driver del database il compito di sanificare i parametri, rendendo le iniezioni SQL strutturalmente impossibili.
- **Validazione Upload Sicura:** Nell'area amministrativa, il sistema di caricamento delle immagini per le notizie non si limita a controllare l'estensione del file (es. `.jpg` o `.webp`). Sfruttiamo controlli MIME avanzati tramite `getimagesize()` per verificare che il file sia realmente un'immagine e non uno script PHP mascherato, bloccando alla radice potenziali intrusioni nel server.

5.2.4 Accessibilità generata Server-Side e Gestione Errori

Abbiamo integrato l'accessibilità direttamente nel motore PHP, evitando di delegare la modifica degli stati visivi solo a JavaScript. Il backend calcola e inietta gli attributi ARIA appropriati; ad esempio, se le query di `TicketManager` riportano che i posti per una tribuna sono esauriti, il file `abbonamento.php` stampa dinamicamente l'attributo `aria-disabled="true"` e le classi di blocco direttamente nel DOM.

Infine, abbiamo prestato molta attenzione alla gestione degli errori e alle operazioni critiche sul database. I passaggi più delicati, come il salvataggio di un ordine dal carrello, sono inseriti all'interno di blocchi `try-catch`. Se qualcosa va storto durante il checkout, blocchiamo l'operazione facendo un *rollback*, così evitiamo di salvare dati incompleti o corrotti. L'utente viene poi reindirizzato a una pagina sicura con un messaggio di errore chiaro (generato tramite `Tool::buildMessage()`). In questo modo evitiamo che la pagina crashi mostrando i classici errori nativi di PHP, che finirebbero per svelare percorsi del server o dettagli delle query SQL. Con la stessa logica di sicurezza abbiamo gestito il logout: non ci limitiamo a svuotare l'array di sessione sul server, ma andiamo a cancellare fisicamente i cookie dal browser del client, azzerando qualsiasi rischio di riutilizzo della sessione.

5.3 Comportamento Client-Side e Interazioni Asincrone (JavaScript)

Il livello di interazione lato client è stato sviluppato interamente in *Vanilla JavaScript* (ECMAScript 6+), escludendo l'uso di librerie esterne o framework. L'obiettivo è stato quello di arricchire l'esperienza utente mantenendo il caricamento delle pagine rapido e il codice manutenibile.

L'architettura JavaScript del progetto si fonda su quattro pilastri:

- **Comunicazione Asincrona (AJAX e Fetch API):** Per evitare ricaricamenti di pagina durante le operazioni di routine, abbiamo deciso di utilizzare chiamate asincrone tramite la `Fetch API`. Un esempio è la rimozione di un biglietto nel `carrello.js`: quando l'utente elimina un elemento, il client invia una richiesta POST al server con l'indice del biglietto; alla ricezione della conferma (in formato JSON), il JavaScript aggiorna il totale a schermo e rimuove l'elemento dal DOM applicando un'animazione CSS, senza interrompere la navigazione.
- **Gestione Dinamica delle Viste:** In sezioni complesse come l'Area Admin o l'Area Utente, invece di avere file HTML separati per visualizzare la lista delle FAQ o il form per inserirne una nuova, il JavaScript intercetta i click sui menu laterali e "accende o spegne" le diverse sezioni della pagina manipolando le classi CSS (`.active` o `.hidden`). Questo approccio azzerava i tempi di caricamento durante la navigazione interna dei pannelli di controllo.
- **Controllo dei form in tempo reale e gestione errori:** Nel file `validazione-reg.js` volevamo dare un aiuto immediato a chi si registra, senza aspettare che preme il tasto "Invia". Mentre l'utente scrive la password, usiamo delle espressioni regolari per verificare subito i requisiti (lettere maiuscole, numeri, caratteri speciali) e aggiorniamo una checklist a schermo che si colora man mano che le regole vengono rispettate. Abbiamo curato anche l'accessibilità dei messaggi d'errore asincroni (le chiamate AJAX): quando qualcosa va storto, il JavaScript crea al volo un blocco con l'attributo `aria-live="assertive"`. In questo modo forziamo lo screen reader ad annunciare subito il problema, avvisando tempestivamente l'utente.
- **Gestione del menu da tastiera:** Nello script `nav.js` abbiamo curato l'accessibilità del menu "hamburger" per mobile. Abbiamo aggiunto un ascoltatore per l'evento `keydown` in modo che, se l'utente preme il tasto `Escape` con la tendina aperta, il menu si chiude in automatico e l'attributo `aria-expanded` torna a "false". La cosa più importante che abbiamo implementato è il ritorno del *focus*: una volta chiuso il menu, il cursore torna esattamente sul pulsante di apertura. In questo modo, chi naviga solo con la tastiera non rischia di ritrovarsi sbalzato all'inizio o in fondo alla pagina.
- **Dark Mode e salvataggio delle preferenze:** Nel file `theme.js` abbiamo gestito il tema scuro evitando di manipolare gli stili inline tramite JavaScript. Lo script si limita ad applicare l'attributo `data-theme="dark"` direttamente al tag `<html>`, lasciando al CSS il compito di scambiare le variabili colore. La scelta dell'utente viene salvata nel `localStorage` per mantenerla attiva durante tutta la navigazione. Come accortezza in più, per i nuovi visitatori, utilizziamo `window.matchMedia` per leggere le impostazioni del sistema operativo (ad esempio, se il telefono o il PC dell'utente è già in modalità scura) e adattiamo il sito di conseguenza fin dal primo caricamento.

5.4 Sviluppo del Livello di Presentazione (CSS3 e Design System)

Il foglio di stile del progetto è stato sviluppato adottando un approccio architetturale solido e moderno, senza l'ausilio di framework esterni, per garantire il massimo controllo sulle performance e sulla personalizzazione visiva.

5.4.1 Design System e CSS Custom Properties

L'intera interfaccia si basa su un Design System centralizzato governato da CSS Custom Properties (variabili) dichiarate nella pseudo-classe `:root`. Questa scelta ha permesso di tokenizzare l'intera palette cromatica (suddivisa in colori di superficie, testo e accenti), le spaziature fluide e i livelli di opacità precompilati (es. `-col-shadow-15`). L'utilizzo dei token ha garantito una totale coerenza visiva lungo tutte le macro-sezioni dell'applicativo, facilitando future operazioni di manutenzione.

5.4.2 Dark Mode Architetturale

L'implementazione del tema scuro (*Dark Mode*) è stata progettata per essere scalabile e priva di ridondanze strutturali. Invece di riscrivere le regole CSS per ogni singolo componente, il tema scuro viene attivato a livello globale mediante l'attributo `[data-theme="dark"]`, il quale si limita a sovrascrivere i valori delle variabili radice (come `-col-bg`, `-col-surface` e le palette dei testi). Di conseguenza, tutti i moduli ereditano dinamicamente i nuovi contrasti, garantendo un'esperienza immersiva al buio senza incrementare il debito tecnico o il peso del file.

5.4.3 Accessibilità Visiva e Media Queries Avanzate

In totale conformità con le direttive WCAG, un'attenzione particolare è stata posta all'inclusività motoria e visiva del front-end:

- **Gestione del Focus:** Tutti gli elementi interattivi (pulsanti, link, controlli dei form) presentano stati di `:focus-visible` rigorosamente standardizzati, i quali applicano un `outline` marcato e spaziatto per guidare agevolmente la navigazione da tastiera.
- **Animazioni Ridotte (`prefers-reduced-motion`):** Sono state integrate speciali *media queries* in grado di interrogare le preferenze di sistema dell'utente. Qualora l'utente abbia richiesto una riduzione del movimento (per prevenire disagi vestibolari), il CSS disabilita nativamente tutte le transizioni, le animazioni e lo scorrimento fluido (`scroll-behavior: auto`).

5.4.4 Approccio DRY e Raggruppamento dei Componenti

Per ottimizzare la pulizia del codice e scongiurare le ripetizioni (rispettando il pattern architetturale DRY, *Don't Repeat Yourself*), le regole strutturali condivise sono state consolidate in macro-selettori di raggruppamento. Le proprietà grafiche identiche, come `border-radius`, transizioni e `box-shadow` delle *card* oppure i *padding* e i colori di sfondo dei pulsanti, sono centralizzate. Le classi specifiche di ciascun componente si limitano a definire unicamente le logiche interne di layout (tramite *CSS Grid* e *Flexbox*), mantenendo un'architettura a componenti estremamente robusta.

5.4.5 Ottimizzazione per la Stampa

Il livello di presentazione è stato arricchito con regole mirate per la corretta impaginazione e la fruizione su supporti cartacei. Il layout di stampa agisce in modo sottrattivo e normalizzante:

- Gli elementi interattivi non essenziali per la lettura (come il menu di navigazione, i pulsanti d'acquisto, la sidebar utente e lo *skip-link*) vengono nascosti automaticamente tramite la direttiva `display: none`.
- Il foglio di stile neutralizza le ombreggiature e disabilita i background colorati, forzando un alto contrasto basato su testo nero puro (`var(-nero-puro)`) su sfondi bianchi (`var(-bianco-puro)`).

- Per prevenire il troncamento dei testi a metà tra due pagine stampate, i contenitori principali (come i riquadri delle date, i biglietti e gli elementi *FAQ*) fanno uso delle direttive `page-break-inside: avoid` e `break-inside: avoid`.
- I collegamenti ipertestuali sono stati espansi tramite l'uso degli pseudo-elementi (`::after` e `attr(href)`), permettendo di stampare fisicamente l'URL di destinazione accanto al testo del link.

6 Installazione e Configurazione del Sistema

6.1 Ambiente di sviluppo utilizzato

Per lo sviluppo e l'esecuzione del progetto abbiamo utilizzato l'ambiente **MAMP**, una soluzione integrata che racchiude in un unico pacchetto i tre componenti necessari al funzionamento di un'applicazione web dinamica:

- un **server web Apache**, che ospita i file del sito all'interno della cartella `htdocs`;
- un **interprete PHP**, per l'esecuzione della logica lato server;
- un **database MySQL/MariaDB**, per la persistenza dei dati, amministrabile comodamente tramite l'interfaccia `phpMyAdmin` inclusa.

L'intero applicativo è stato sviluppato senza framework esterni, ricorrendo unicamente alle tecnologie standard: HTML5, CSS3, JavaScript (*Vanilla*, ECMAScript 6+) e PHP.

6.2 Procedura di installazione

Per installare ed eseguire il progetto in locale è sufficiente seguire i passaggi descritti di seguito:

1. **Posizionamento dei file:** copiare la cartella del progetto all'interno della directory `htdocs` di MAMP, quindi avviare i servizi *Apache* e *MySQL* dal pannello di controllo.
2. **Creazione e importazione del database:** accedere a `phpMyAdmin`, creare un nuovo database denominato `pataviumopen` e importarvi l'unico file `database/patavium_open.sql`, che si occupa di creare tutte le tabelle (con i relativi vincoli) e di popolarle con i dati iniziali (incontri, FAQ, campioni, ecc.).
3. **Configurazione delle credenziali:** le credenziali di accesso al database non sono salvate direttamente nel codice versionato. Occorre quindi creare il file di configurazione locale copiando il template fornito `config/database.example.php` in un nuovo file `config/database.local.php` e inserendovi i valori corretti per il proprio ambiente. Con un'installazione MAMP standard la configurazione è la seguente:

```
return [
    'host' => 'localhost',
    'user' => 'root',
    'pass' => 'root',
    'name' => 'pataviumopen',
];
```

Il file `database.local.php` è volutamente escluso dal versionamento (tramite `.gitignore`): in questo modo ogni sviluppatore può mantenere le proprie credenziali reali senza esporle nel repository, mentre il template `database.example.php` resta versionato come riferimento.

4. **Avvio dell'applicazione:** aprire il browser e raggiungere il progetto all'indirizzo locale (ad esempio `http://localhost/progetto_tecweb_2026/`). Il file `index.php` in radice gestisce l'instradamento verso la Home Page del portale.

Per facilitare la valutazione, il sistema è già predisposto con due utenze di prova: un account **amministratore** (username `admin`, password `admin`) per accedere al pannello di gestione e un account **utente** standard (username `user`, password `user`) per provare il flusso di acquisto e il sistema di supporto.

7 Metodologia di Lavoro e Suddivisione dei Compiti

Il lavoro di sviluppo è stato portato avanti dal nostro gruppo composto da quattro persone: Anna Marini, Eleonora Bellet, Lorenzo Milanese e Diego Belluz.

7.1 Pianificazione e Organizzazione

Data l'importanza di prendere decisioni condivise fin da subito, le fasi iniziali di *brainstorming* e la definizione dello schema ER (il modello del database) si sono svolte tramite sessioni di confronto su Discord. Questa scelta è stata cruciale perché ci ha permesso di decidere insieme la struttura del database e l'architettura generale del sito prima di scrivere anche una sola riga di codice.

Durante questi incontri abbiamo anche definito delle regole comuni di base per non "incasinarci" durante lo sviluppo:

- **Naming Convention:** abbiamo deciso di usare esclusivamente il formato `snake_case` per i nomi di tutti i file.
- **Struttura delle cartelle:** abbiamo creato una suddivisione chiara tra i file PHP e i file HTML per tenere tutto in ordine.

7.2 Suddivisione dello sviluppo

Per lavorare in modo efficiente e parallelo, abbiamo deciso di non metterci mai a lavorare sugli stessi file contemporaneamente, evitando così conflitti e sovrapposizioni.

7.2.1 Fase 1: Struttura HTML e Database

Mentre tre di noi (Eleonora, Diego e Anna) si dedicavano alla costruzione dell'HTML, Lorenzo si è occupato del *backend* inteso come base dati, creando il file `patavium_open.sql` e gestendo la logica del database. Gli altri tre componenti hanno suddiviso le pagine del sito: chi si occupava della sezione "Biglietti" ha gestito sia la pagina principale che la creazione delle *card* e degli *item* specifici (come le sessioni e i Ground Pass), così da mantenere omogeneo lo stile di ogni singola pagina.

7.2.2 Fase 2: Logica PHP e JavaScript

Una volta completata la parte strutturale, ci siamo spostati sulla logica vera e propria. Anche in questa fase abbiamo lavorato in parallelo: ognuno di noi ha preso in carico dei moduli specifici. Ad esempio, chi aveva già sviluppato la pagina Biglietti ha curato anche la logica di inserimento nel carrello, mentre altri si sono concentrati sulla gestione delle FAQ o sull'Area Utente.

Per quanto riguarda JavaScript e PHP, abbiamo evitato di scrivere codice sparso: le funzioni più complesse o riutilizzabili sono state create da chi di noi era più focalizzato su quella specifica funzionalità, per poi essere condivise e richiamate da tutti. Questo ci ha permesso di non dover

fare ognuno il "proprio" PHP o il "proprio" JS, ma di avere un codice che funzionasse in modo uguale su tutto il sito.

Abbiamo inoltre suddiviso in modo **equo** il carico di lavoro tra le due tecnologie, in modo che nessuno si occupasse esclusivamente del backend o esclusivamente dell'interattività lato client. A titolo indicativo, la ripartizione dei moduli principali è stata la seguente:

- **Anna Marini** ha curato la logica PHP della registrazione e dell'autenticazione (`account_manager.php`, validatori della classe `Tool`) e, lato JavaScript, la validazione in tempo reale del form di registrazione (`validazione-reg.js`) e la gestione del tema scuro (`theme.js`).
- **Eleonora Bellet** ha sviluppato la logica PHP della biglietteria e del carrello (`carrello_manager.php`, `checkout.php`) e, lato JavaScript, le chiamate asincrone di aggiunta e rimozione dei biglietti (`script.js`, `carrello.js`).
- **Diego Belluz** si è occupato del sistema di supporto e FAQ lato PHP (`faq_manager.php`, `ticket_manager.php`) e, lato JavaScript, della barra di ricerca delle FAQ (`faq_search.js`) e dell'accessibilità del menu da tastiera (`nav.js`).
- **Lorenzo Milanese** ha realizzato l'intera area amministrativa lato PHP (`news_manager.php`, `campioni_manager.php` e l'orchestrazione di `area_admin.php`) e si è occupato fin dall'inizio della progettazione del database, scrivendo lo schema e i dati iniziali nel file `patavium_open.sql`. Lato JavaScript ha sviluppato tutta la gestione dinamica del pannello admin (`gestione_news.js`, `gestione_campioni.js`, `gestione_faq.js`), la gestione delle risposte ai quesiti degli utenti (`gestione_domande.js` e `gestione_risposte_utente.js`) e l'indicatore delle pagine visitate con la pallina da tennis (`visited-links.js`). Ha inoltre curato l'ottimizzazione del foglio di stile per la stampa (`print.css`).

In questo modo ognuno ha scritto sia codice PHP sia codice JavaScript, prendendo confidenza con entrambi i livelli dello stack.

Il foglio di stile (CSS), invece, non è stato assegnato a una sola persona: lo abbiamo scritto un po' tutti. Ciascuno si è occupato della grafica delle pagine e dei componenti su cui stava lavorando, appoggiandosi però a un insieme di variabili e di classi comuni (i token del Design System) che avevamo concordato insieme all'inizio. In questo modo abbiamo mantenuto uno stile coerente in tutto il sito ed evitato che ognuno reinventasse a modo proprio spaziature, colori o bordi.

Questo approccio ci ha permesso di procedere in parallelo, completando le varie aree funzionali (come il carrello, l'area admin e il sistema di notifiche) in tempi brevi, riuscendo a integrare tutto correttamente grazie al continuo allineamento durante le nostre sessioni di lavoro.

7.3 Strumenti di comunicazione

Per mantenere il team sempre allineato e gestire le diverse fasi di sviluppo in modo ordinato, abbiamo adottato una strategia basata su strumenti di comunicazione e collaborazione immediati, evitando complessità eccessive che avrebbero potuto rallentare il lavoro.

La comunicazione quotidiana si è basata principalmente su due pilastri:

- **Discord:** utilizzato per le chiamate di gruppo e per le sessioni di brainstorming più lunghe. È stato lo spazio virtuale in cui abbiamo preso le decisioni più importanti, come la definizione del database e la struttura delle cartelle.
- **WhatsApp:** utilizzato per scambi di informazioni rapide, alert immediati e per mantenere un contatto costante e informale durante le ore di lavoro individuale.

7.4 Monitoraggio dei Task: La “Task Board” su Google Doc

Per tracciare lo stato di avanzamento del lavoro, abbiamo creato un documento Google condiviso contenente una tabella di gestione dei task. L’idea di fondo era quella di avere uno strumento simile a un sistema di *Issue Tracking* (come quello di GitHub), ma più semplice e immediato da consultare per tutti i membri del gruppo.

La tabella era strutturata in modo molto chiaro:

- **Obiettivo:** una breve descrizione del task (es. “Creazione pagina FAQ”, “Validazione form Login”).
- **Responsabile:** il nome del componente del team incaricato di completare il compito.
- **Stato:** un campo dinamico che aggiornavamo costantemente con tre stati: *Da fare*, *In corso*, *Completato*.

Questa “bacheca” digitale ci ha permesso di avere sempre una visione d’insieme chiara di cosa mancava, chi stava lavorando su cosa e a che punto era il progetto. È stata la nostra soluzione per evitare che due persone lavorassero sullo stesso task senza saperlo e per assicurarci di arrivare alla consegna con tutti i pezzi pronti e testati.